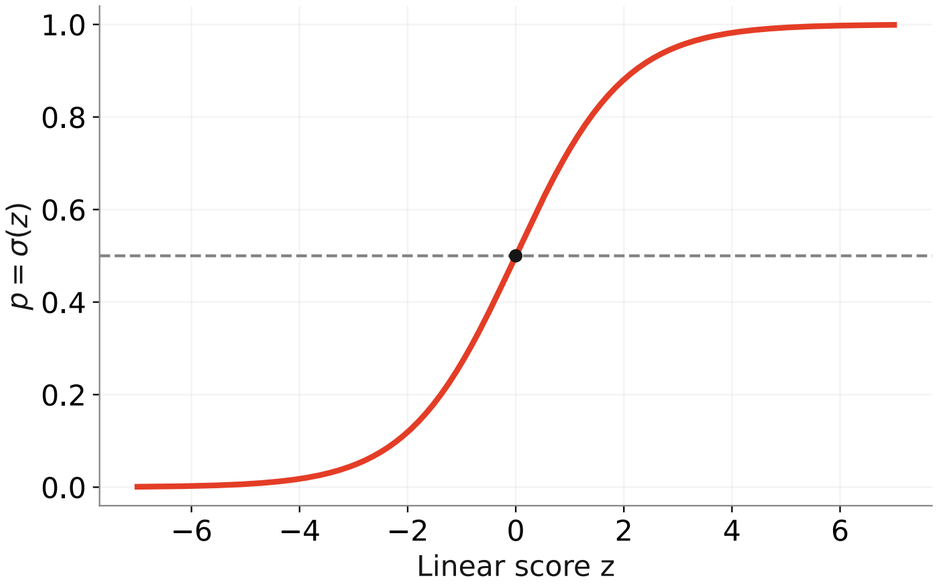
From regression to classification.

How models learn, estimate probabilities, and turn predictions into decisions.



Model → loss → gradient → evaluation → decision.

Three milestones, one learning problem

Fitting a line, estimating a class probability and training a neural network all require parameters to be learned from observations.

The central question: **which parameters explain the observations and predict new cases well?**

01 **1805 · Legendre**
Least squares: combine noisy observations by minimising squared errors.

02 **1958 · Cox**
Binary regression: model how the probability of 1 depends on predictors.

03 **1986 · Rumelhart, Hinton & Williams**
Backpropagation: organise the chain rule across multilayer networks.

Model → objective function → optimisation → prediction.

Specify the prediction before choosing a model

Each labelled example contains an input x_i and a known target. The index i identifies one observation; n is the training-set size.

Our running example: distance x (km) and walking duration y (min).

Parameters w, b are learned from examples. A feature must be available when a prediction is requested.

Task	Target	Output
Regression	$y \in \mathbb{R}$	Duration
Classification	$c \in \{0, 1\}$	Over 30 min?

$$\hat{y} = wx + b$$

$$p = P(c = 1 \mid x)$$

A numerical target and a class label require different output interpretations.

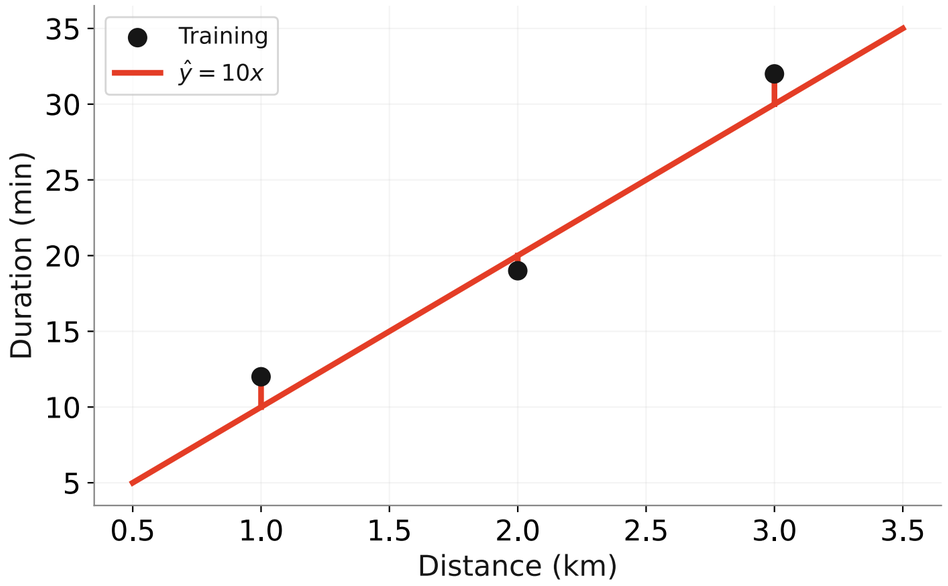
Linear regression: make “best fit” precise

The model predicts $\hat{y}_i = wx_i + b$. Define the signed residual as prediction minus observation: $r_i = \hat{y}_i - y_i$.

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^{n} r_i^2$$

Squaring prevents positive and negative errors from cancelling, and gives large errors more weight.

$J = \mathrm{MSE}/2$. The factor $1/2$ simplifies derivatives without changing the minimiser.



At $w = 10, b = 0$, residuals are $(-2, 1, -2)$ min; $J = (4 + 1 + 4)/6 = 1.5$ min².

The loss turns “fit the data” into an explicit optimisation objective.

The gradient follows the computation

For one observation, $\ell_i = r_i^2/2$. A derivative measures how the loss changes when a parameter changes slightly.

$$\frac{\partial \ell_i}{\partial w} = \frac{\partial \ell_i}{\partial r_i}\frac{\partial r_i}{\partial w} = r_i x_i$$

$$\frac{\partial \ell_i}{\partial b} = r_i \cdot 1$$

The derivative of a sum is the sum of the derivatives. Average the contributions of all training observations.

01 Forward

$w, b \longrightarrow \hat{y}_i \longrightarrow r_i \longrightarrow \ell_i$

02 Differentiate

Along the w path: multiply r_i by x_i. Along the b path: multiply by 1.

$$g_w = \frac{1}{n}\sum_i r_i x_i$$

$$g_b = \frac{1}{n}\sum_i r_i$$

The chain rule connects local derivatives to a parameter's effect on the loss.

One update, computed by hand

Start at $w = 10, b = 0$. Compute both gradients from these same old parameters.

x_i	y_i	r_i	$r_i x_i$
1	12	−2	−2
2	19	1	2
3	32	−2	−6

$$g_w = -6/3 = -2, \qquad g_b = -3/3 = -1$$

$$w^+ = w - \eta g_w, \quad b^+ = b - \eta g_b$$

With learning rate $\eta = 0.1$:

$$w^+ = 10.2, \qquad b^+ = 0.1$$

The new residuals are $(-1.7, 1.5, -1.3)$.

$$J^+ = 6.83/6 \approx 1.1383 < 1.5$$

Subtract the gradient; update all parameters simultaneously.

An exact solution checks the iteration

At a differentiable minimum, both partial derivatives vanish. For this dataset:

$$14w + 6b = 146$$

$$6w + 3b = 63$$

The loss surface is bowl-shaped (convex), with no spurious local minima. Distinct distances make this stationary point the unique global minimum.

$$w_* = 10, \qquad b_* = 1$$

$$\hat{y} = 10x + 1, \qquad J_* = 1$$

01 **Interpretation**

Slope: 10 min/km. Intercept: 1 min, a fitted offset.

02 **Geometry**

At the optimum, $\sum_i r_i = 0$ and $\sum_i r_i x_i = 0$. Residuals balance along the fitted directions.

Least squares can be solved directly; iterative optimisation generalises to richer models.

The step size controls learning dynamics

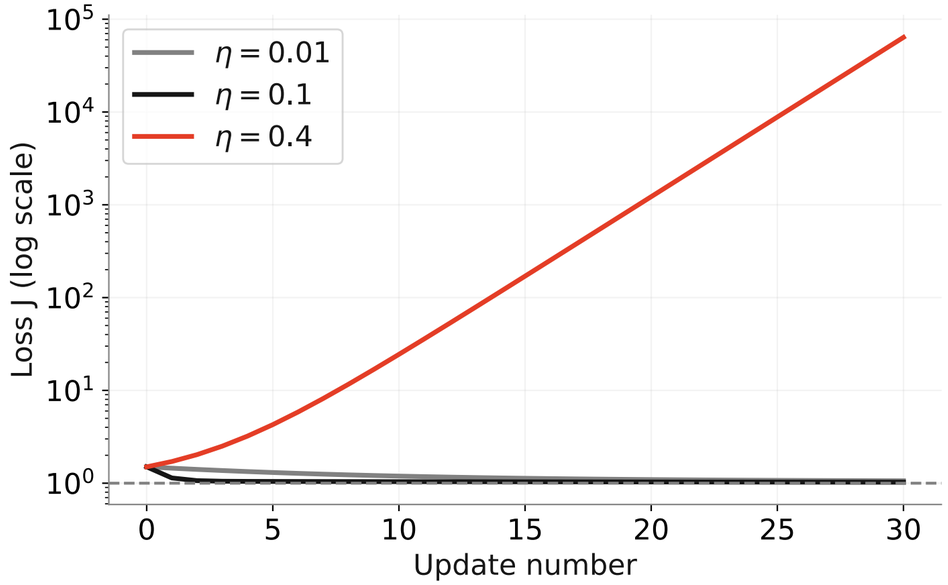
The gradient chooses a locally downhill direction. The learning rate η determines how far to move.

01 Too small
Progress may be slow.

02 Appropriate
The loss decreases towards the minimum.

03 Too large
The update can overshoot and diverge.

Feature units affect gradient magnitudes and useful step sizes.



Same observations and start; only the learning rate changes. The vertical axis is logarithmic.

A valid gradient does not guarantee that every finite step decreases the loss.

Training is a loop; inference is an evaluation

01 Initialise
Choose parameters, a loss, a learning rate and a stopping rule.

02 Predict and score
Compute predictions and mean training loss.

03 Differentiate and update
Compute all gradients, then update the parameters.

04 Repeat
Stop using the predefined rule; keep the resulting parameters.

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla J\left(\theta^{(t)}\right)$$

Here $\theta = (w, b)$, t counts updates, and ∇J collects the partial derivatives.

At inference, keep parameters fixed and evaluate the learned function on a new input.

$$x_{\text{new}} = 2.4 \implies \hat{y} = 25 \text{ min}$$

The learning algorithm creates a prediction function that can later be reused.

Three datasets, three distinct questions

Low training loss describes cases already used to fit the model. Our goal concerns future cases.

Split before fitting. The split should reflect deployment: future dates or new users may require temporal or grouped splits.

Repeatedly inspecting test results turns the test set into another source of selection.

01 Training · fit
Estimate model parameters using the training loss.

02 Validation · select
Choose complexity, hyperparameters and decision threshold.

03 Test · report
After choices are frozen, estimate performance on untouched examples.

Generalisation is measured on data that did not guide fitting or selection.

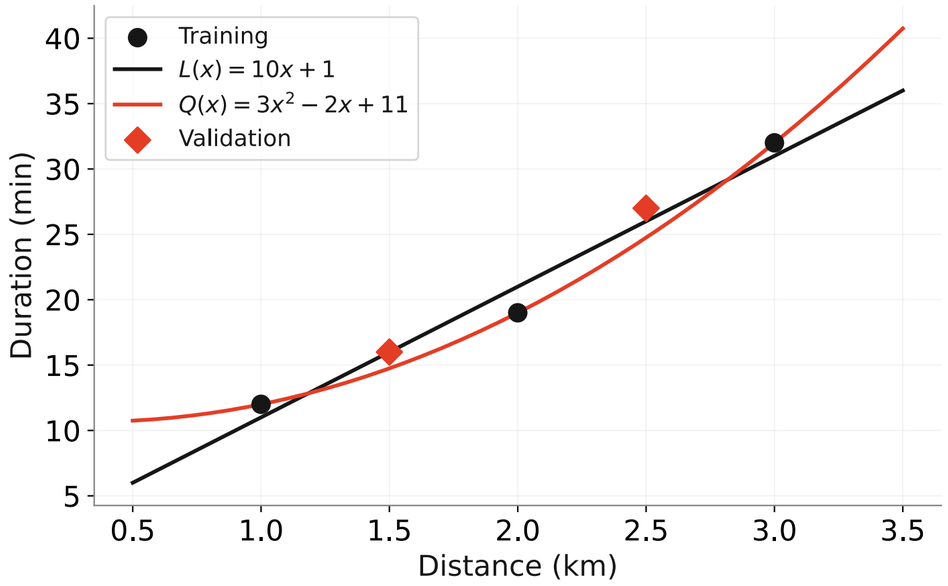
A perfect training fit can lose on new data

Compare candidates fitted to the same three training observations, using mean squared error (MSE).

Model	Train MSE	Validation MSE
Line L	2	0.5
Quadratic Q	0	3.3125

Validation: $(1.5, 16)$ and $(2.5, 27)$. L predicts $(16, 26)$; Q predicts $(14.75, 24.75)$.

Select the line using validation. A separate test is still needed.



Higher capacity can explain training fluctuations. These tiny datasets illustrate the protocol, not reliable performance estimates.

Optimisation asks whether we fitted the data; evaluation asks whether the fit transfers.

Use information available at prediction time

Distance is known before a walk. Arrival time reveals the outcome after the walk and cannot serve as an input here.

Preprocessing also learns from data. Estimate means and standard deviations on training data only.

Reuse those same values for validation, test and future inputs.

$$x' = \frac{x - \mu_{\text{train}}}{s_{\text{train}}}$$

01 **Learn on training**
Estimate μ_{train} and $s_{\text{train}} > 0$.

02 **Transform everywhere**
Reuse the fixed transformation.

03 **Preserve the protocol**
Selecting a transformation is a validation decision.

Data leakage lets information about the answer enter the prediction procedure.

Classification predicts categories

A classification target is a category. Its numerical code identifies a class; it does not represent a measured quantity.

01 Binary
Two mutually exclusive outcomes: spam or legitimate email.

02 Multiclass
One of several classes: the digit 0, 1, ..., 9.

03 Multilabel
Several labels may apply: a photo can contain both a person and a bicycle.

$$\text{binary}: \quad c \in \{0, 1\}$$

$$\text{multiclass}: \quad c \in \{1, \ldots, K\}$$

$$\text{multilabel}: \quad \mathbf{c} \in \{0, 1\}^K$$

Learn a binary probability model, convert its output into a class decision, then evaluate the decisions.

Logistic regression is a classification model despite the word “regression” in its name.

Task → probability model → class decision → classification evaluation.

Will the walk exceed 30 minutes?

Define $c = 1$ when the measured duration exceeds 30 min; otherwise $c = 0$.

$$p(x) = P(c = 1 \mid x)$$

Distance is still the input. The output now estimates the chance of an event.

Unobserved factors such as speed allow different outcomes at the same distance.

Distance x	Duration y	Class c
1	12	0
2	19	0
3	32	1
4	29	0
5	48	1
6	57	1

A probability prediction and a binary decision are two separate outputs.

Give the linear score a probability interpretation

A raw score $z = wx + b$ can be any real number. A probability must lie between 0 and 1.

We need a smooth map from score to probability so that parameter changes can still be differentiated.

Clipping a line at 0 and 1 creates flat regions and does not specify a probabilistic training objective.

01 Feature

$x \in \mathbb{R}$

02 Linear score

$z = wx + b \in \mathbb{R}$

03 Probability

$p = \sigma(z) \in (0, 1)$

04 Decision later

Compare p with a threshold τ.

The score combines features; the probability link gives it an interpretable scale.

Derive the sigmoid from linear log-odds

For probability p, the odds are $p/(1-p)$. At $p=0.75$, the odds are $3:1$.

Logistic regression assumes that the logarithm of the odds depends linearly on the features. This is a modelling assumption.

$$\log \frac{p}{1-p} = z = wx + b$$

Here and throughout, log means natural logarithm.

$$\frac{p}{1-p} = e^z$$

$$p = e^z(1-p)$$

$$p(1+e^z) = e^z$$

$$p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

The last expression defines the logistic sigmoid.

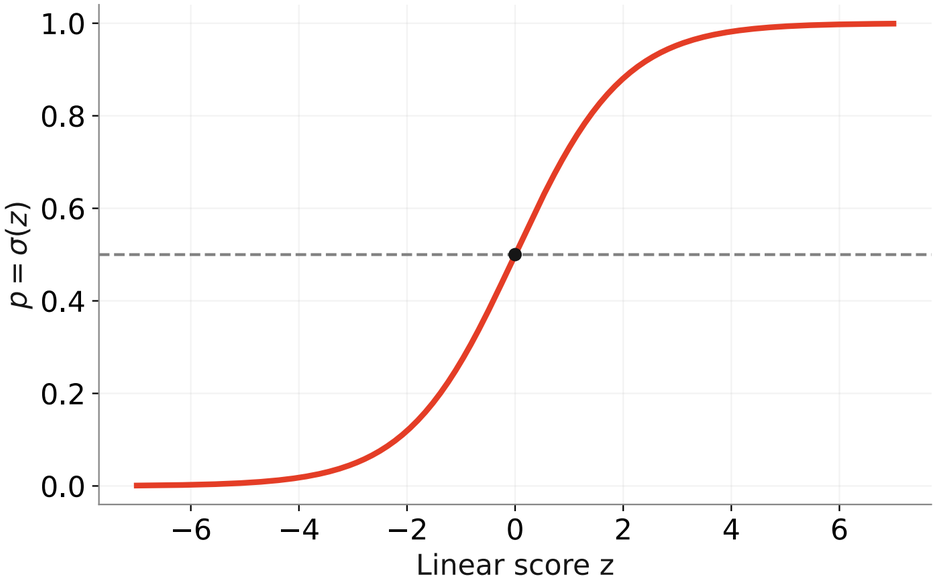
The sigmoid follows algebraically once linear log-odds have been assumed.

Read the sigmoid as a probability scale

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Score	Probability
$z = 0$	$p = 0.5$
$z \to +\infty$	$p \to 1$
$z \to -\infty$	$p \to 0$

The curve is continuous and strictly increasing. Larger scores imply larger estimated probabilities of class 1.



Symmetry: $\sigma(-z) = 1 - \sigma(z)$. The values 0 and 1 are never reached for finite scores.

A probability near 0.5 expresses uncertainty within the model.

Compute a probability before making a decision

For exact arithmetic, take illustrative parameters $w = \log 3$ and $b = -3\log 3$. These parameters have not been fitted.

$$z = (x - 3)\log 3$$

At $x = 4$ km, $z = \log 3$. The odds are 3 and the probability is $3/4$.

x	z	p
2	$-\log 3$	0.25
3	0	0.50
4	$\log 3$	0.75

A probability of 0.75 estimates the event “duration exceeds 30 min”. It does not guarantee the outcome of this walk.

We still need an objective to learn parameters from labels.

The probability is a model output; its reliability must be checked on data.

Make the observed labels likely

A Bernoulli model has $P(c=1 \mid x)=p$ and $P(c=0 \mid x)=1-p$.

$$P(c_i \mid x_i)=p_i^{c_i}(1-p_i)^{1-c_i}$$

Check both cases: label 1 returns p; label 0 returns 1–p.

Assume conditionally independent labels given inputs and parameters. Their joint likelihood is:

$$L(w,b)=\prod_{i=1}^{n} p_i^{c_i}(1-p_i)^{1-c_i}$$

The observed data are fixed. Likelihood is a function of the parameters.

Maximum likelihood chooses the parameters that maximise this joint value.

Likelihood uses the probability assigned to the labels actually observed.

From likelihood to binary cross-entropy

Logarithms turn a product into a sum. Because log is increasing, maximising likelihood or log-likelihood gives the same solution.

Negating changes maximisation into minimisation. Dividing by n gives a mean loss.

$$J = -\frac{1}{n}\log L$$

$$\ell(p, c) = -c\log p - (1-c)\log(1-p)$$

$$J(w, b) = \frac{1}{n}\sum_{i=1}^{n}\ell(p_i, c_i)$$

Observed label	Loss
c = 1	$-\log p$
c = 0	$-\log(1-p)$

Binary cross-entropy (BCE), also called log loss.

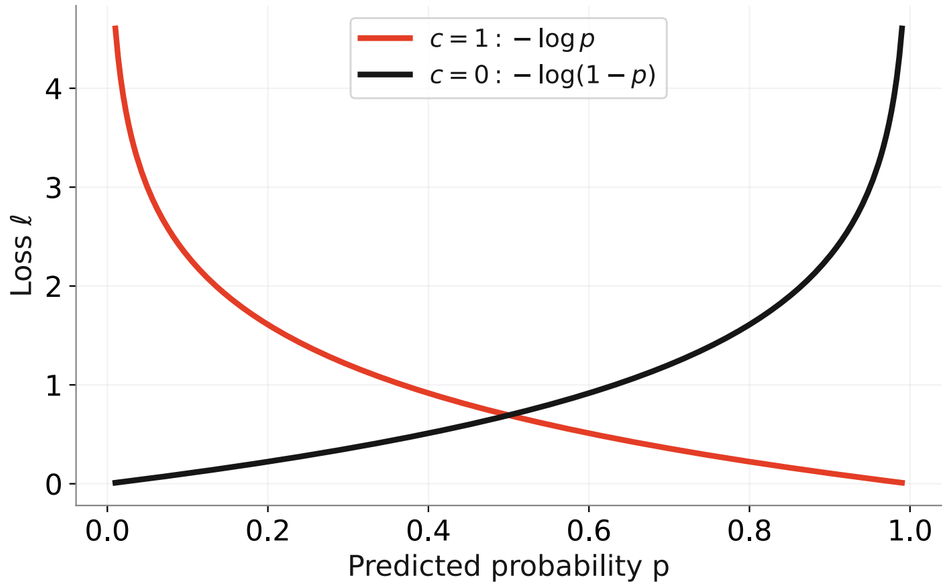
Cross-entropy is the mean negative log-likelihood of the Bernoulli model.

Confident mistakes receive a large penalty

For a true label of 1, increasing the assigned probability lowers the loss. The direction reverses for label 0.

True c	Predicted p	Loss
1	0.9	0.105
1	0.5	0.693
1	0.1	2.303

Accuracy treats wrong decisions equally. Cross-entropy also evaluates the probabilities behind those decisions.



Loss becomes very large when the model assigns almost zero probability to the observed outcome.

Training a probability model requires more than counting correct labels.

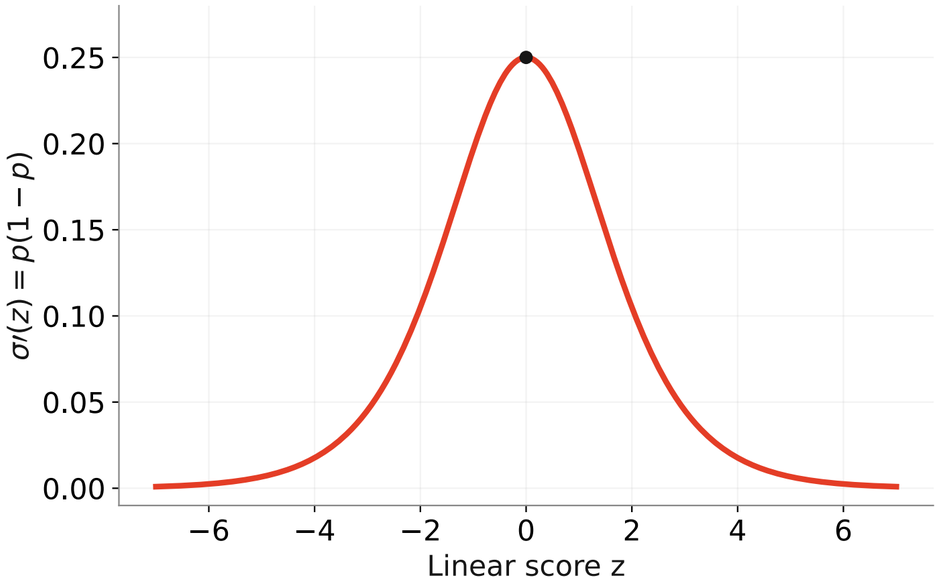
Differentiate the sigmoid first

Write $p = (1 + e^{-z})^{-1}$. Apply the power rule and the chain rule:

$$\frac{dp}{dz} = \frac{e^{-z}}{(1+e^{-z})^2}$$

$$1 - p = \frac{e^{-z}}{1+e^{-z}}$$

$$\frac{dp}{dz} = p(1-p)$$



Maximum slope: $1/4$ at $z = 0$. Far from 0, the probability changes slowly with the score.

This local derivative is one factor in the derivative of the training loss.

The chain rule gives prediction minus label

First differentiate the loss with respect to its probability input:

$$\frac{\partial \ell}{\partial p} = -\frac{c}{p} + \frac{1-c}{1-p}$$
$$= \frac{p-c}{p(1-p)}$$

Then multiply by $dp/dz = p(1-p)$. For finite z, $0 < p < 1$, so cancellation is valid.

$$\frac{\partial \ell}{\partial z} = \frac{p-c}{p(1-p)} p(1-p) = p - c$$

$$\frac{\partial \ell}{\partial w} = (p-c)x$$

$$\frac{\partial \ell}{\partial b} = p - c$$

For c = 1 and p too small, the score derivative is negative: increasing the score reduces the loss.

Sigmoid + cross-entropy produces a particularly simple score gradient.

Average the gradients, then update

$$p_i = \sigma(wx_i + b)$$

$$g_w = \frac{1}{n}\sum_i (p_i - c_i)x_i$$

$$g_b = \frac{1}{n}\sum_i (p_i - c_i)$$

All examples contribute to the batch gradient. One example can pull in a different direction from the average.

01 Predict

Compute scores z_i and probabilities p_i.

02 Compare

Compute $p_i - c_i$ with observed labels.

03 Update

$w^+ = w - \eta g_w, \quad b^+ = b - \eta g_b$

04 Repeat

Monitor loss and use the stopping rule.

The loop stays the same; prediction and loss functions change.

One logistic update on six observations

Start with $w = b = 0$: all probabilities are 0.5 and mean loss is $\log 2 \approx 0.693147$.

x	1	2	3	4	5	6
c	0	0	1	0	1	1
p−c	.5	.5	−.5	.5	−.5	−.5

$$g_b = 0, \qquad g_w = -\frac{7}{12}$$

With $\eta = 0.1$, update both parameters:

$$w^+ = \frac{7}{120} \approx 0.05833, \quad b^+ = 0$$

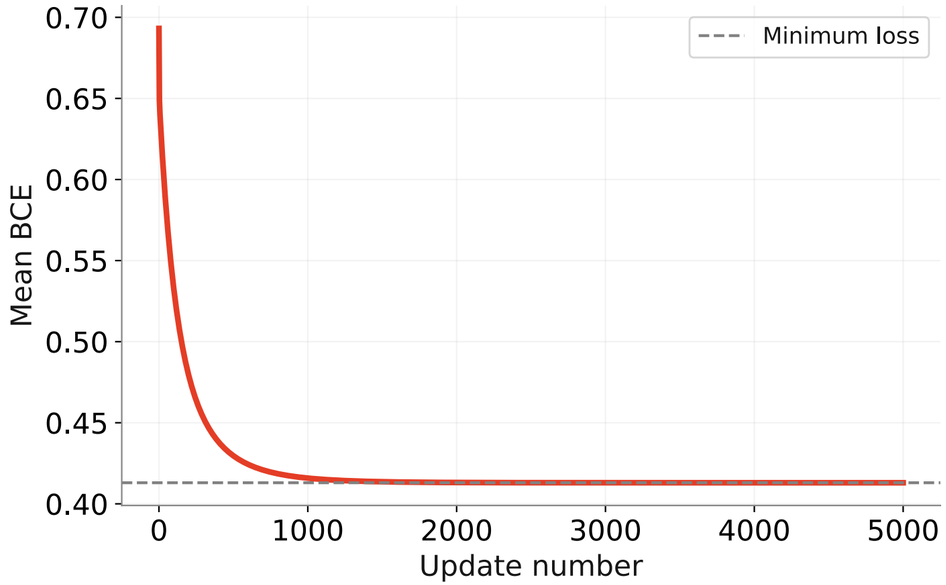
$$p^+(3) = \sigma(0.175) \approx 0.54364$$

The new mean BCE is **0.665548**, below 0.693147.

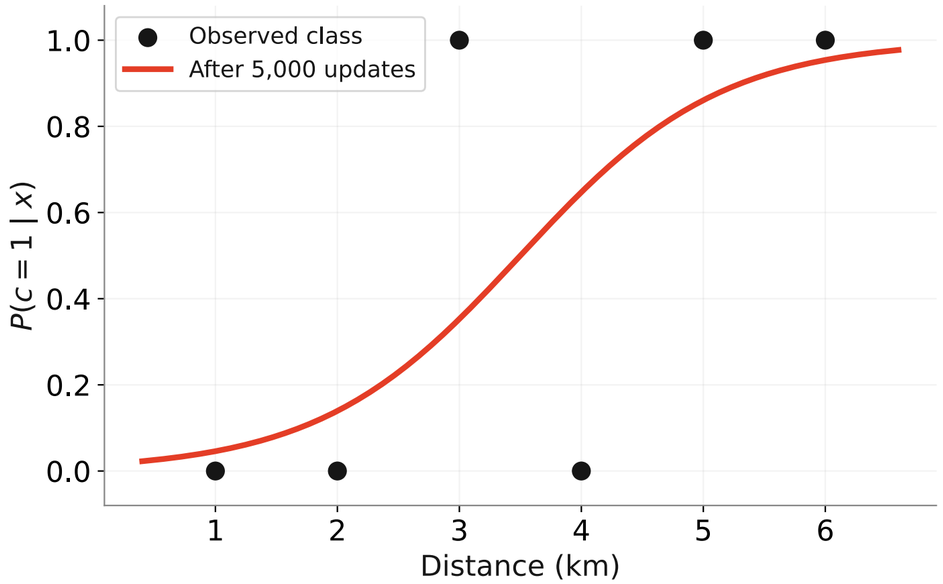
All six probabilities now exceed 0.5. A loss improvement need not improve thresholded accuracy.

The numerical update directly applies the derived gradient.

Repeat the update to fit the probability curve



Batch gradient descent, $\eta = 0.1$, 5,000 updates, start $w = b = 0$.



The labels at x = 3 and x = 4 conflict with a perfect increasing rule. The fitted curve makes a compromise.

Decreasing training loss demonstrates optimisation; new-data evaluation remains necessary.

A threshold turns probability into a decision

Choose $0 < \tau < 1$. Predict class 1 when $p \geq \tau$, class 0 otherwise. Equality is assigned to class 1 throughout this lecture.

$$\hat{c} = 1[p \geq \tau]$$

The indicator is 1 when the statement in brackets is true, and 0 otherwise.

The sigmoid is increasing. A probability threshold is therefore equivalent to a score threshold:

$$p \geq \tau \iff z \geq \log \frac{\tau}{1-\tau}$$

$$\tau = 0.5 \implies wx + b \geq 0$$

With the illustrative parameters on slide 18, the boundary at τ = 0.5 is x = 3 km.

The decision threshold is separate from the learned probability function.

With two features, the boundary is a line

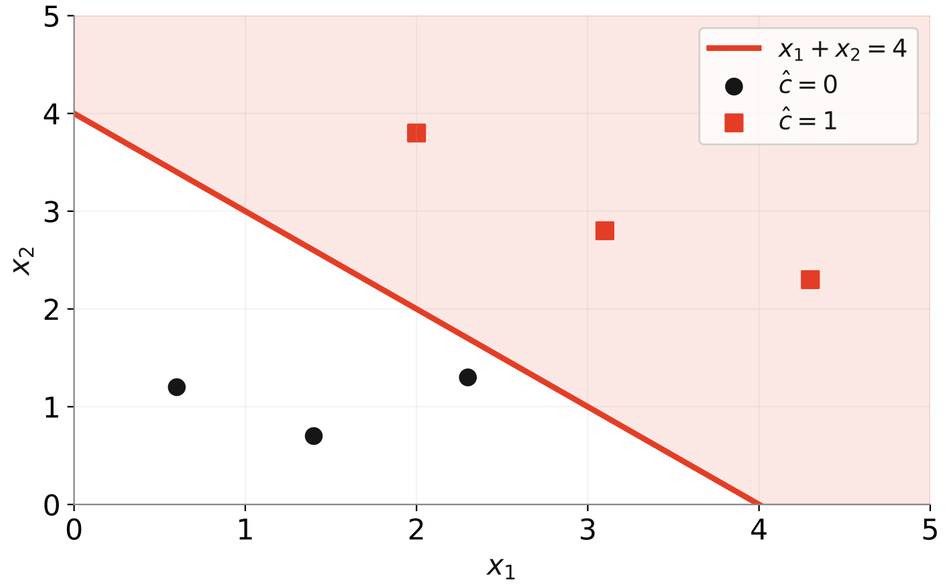
$$z = w_1 x_1 + w_2 x_2 + b$$

$$p = \sigma(z)$$

At $\tau = 0.5$, the boundary is $w_1 x_1 + w_2 x_2 + b = 0$.

The sigmoid changes the score scale. A fixed threshold still gives a linear boundary in the original features.

Illustration: $w_1 = w_2 = 1$, $b = -4$; arbitrary feature units.



Class 1: $x_1 + x_2 \geq 4$. The shaded region shows predicted decisions.

A nonlinear probability curve does not imply a nonlinear decision boundary.

Classification evaluation starts with individual cases

Eight constructed validation cases; predict 1 when $p \geq 0.5$. They are separate from the six training observations.

Case	p	True c	Pred.	Type
A	.95	1	1	TP
B	.85	1	1	TP
C	.70	0	1	FP
D	.60	1	1	TP
E	.45	1	0	FN
F	.30	0	0	TN
G	.20	0	0	TN
H	.10	0	0	TN

Confusion matrix: columns are observed labels; rows are predicted labels.

	Actual 1	Actual 0
Predict 1	TP = 3	FP = 1
Predict 0	FN = 1	TN = 3

TP / TN: correct positive / negative decisions. FP: false alert. FN: missed event.

$$n = TP + FP + FN + TN = 8$$

Specify the positive class and the orientation of the confusion matrix.

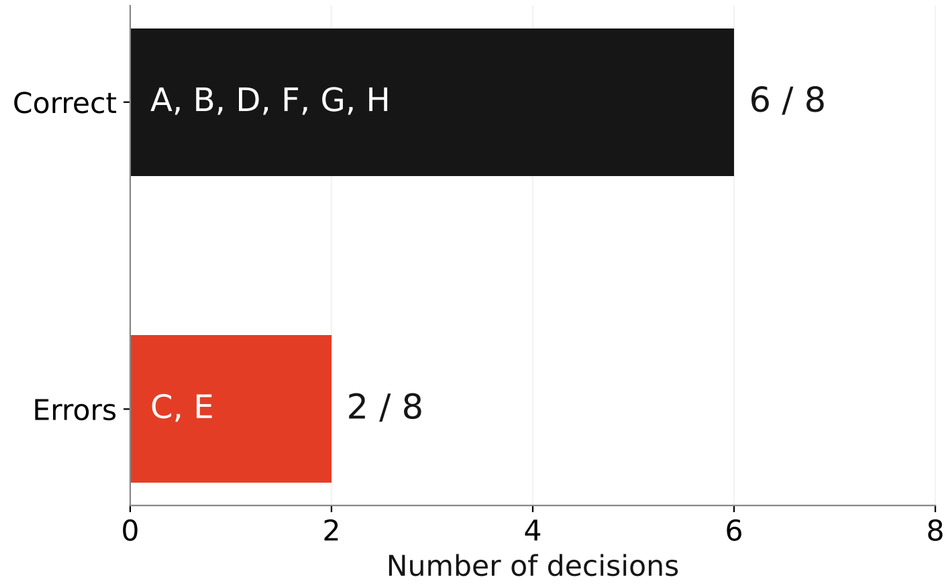
Accuracy: how many decisions are correct?

Count every correct decision, positive or negative, and divide by the total number of cases.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

$$= \frac{3 + 3}{8} = 0.75$$

This measure gives equal weight to every correctly or incorrectly classified case.



Our eight cases: 6 correct decisions and 2 errors. The two errors have different practical meanings.

Accuracy describes overall correctness but does not distinguish false alerts from missed events.

Precision and recall use different denominators

Precision: when the classifier raises an alert, how often is it right?

$$\text{Precision} = \frac{TP}{TP+FP} = \frac{3}{4} = 0.75$$

The predicted-positive group is A, B, C, D. Three of these four alerts are correct.

False positives reduce precision. This matters when reviewing an alert consumes time or resources.

Recall: among the events that really occurred, how many were detected?

$$\text{Recall} = \frac{TP}{TP+FN} = \frac{3}{4} = 0.75$$

The actual-positive group is A, B, D, E. Three of these four events were detected.

False negatives reduce recall. Recall is also called sensitivity or true positive rate (TPR).

Precision starts with predictions; recall starts with observed positives.

F1 combines precision and recall; specificity checks negatives

F1 is the harmonic mean of precision P and recall R. It is high only when both are high.

$$F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$$

$$F_1 = \frac{6}{6+1+1} = 0.75$$

F1 ignores true negatives. It summarises two rates, but does not replace an application-specific cost analysis.

Specificity measures how many actual negatives are correctly left unflagged.

$$\text{Specificity} = \frac{TN}{TN+FP} = \frac{3}{4}$$

$$\text{FPR} = \frac{FP}{FP+TN} = 1 - \text{Specificity}$$

Here specificity = 0.75 and FPR = 0.25. Undefined denominators require an explicitly reported convention.

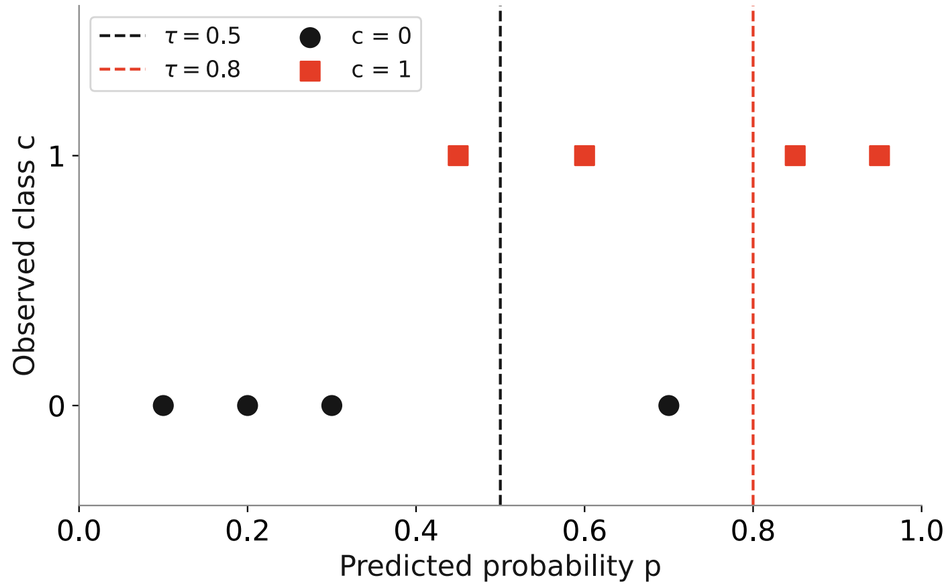
Recall uses actual positives; specificity and FPR use actual negatives.

Change the threshold; change the errors

τ	TP / FP / FN / TN	Precision	Recall
0.5	3 / 1 / 1 / 3	0.75	0.75
0.8	2 / 0 / 2 / 4	1.00	0.50

Both give accuracy 6/8. At 0.8, fewer alerts are issued and more events are missed.

For fixed scores, raising the threshold cannot increase recall. Precision often improves, but can fluctuate.



Scores: .95, .85, .70, .60, .45, .30, .20, .10. Labels: 1, 1, 0, 1, 1, 0, 0, 0.

Select the threshold on validation according to the application; freeze it before testing.

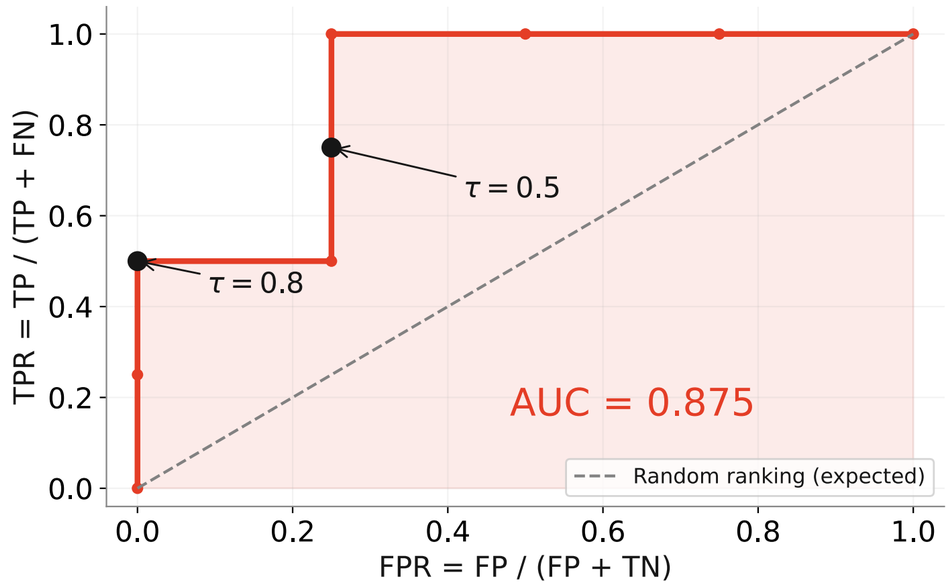
ROC–AUC evaluates ranking across thresholds

Lower the threshold from above every score to below every score. At each step, count TP and FP and plot one point.

$$\mathrm{TPR} = \frac{TP}{TP+FN}, \quad \mathrm{FPR} = \frac{FP}{FP+TN}$$

ROC plots TPR vertically against FPR horizontally. A strong classifier approaches the upper-left corner.

Area under the ROC curve: **AUC = 0.875** here. A random ranking has expected AUC 0.5; a perfect ranking has AUC 1.



Same eight cases. At $\tau = 0.8$: (FPR, TPR) = (0, 0.5). At $\tau = 0.5$: (0.25, 0.75).

AUC summarises ranking; it does not choose a deployment threshold or assess probability calibration.

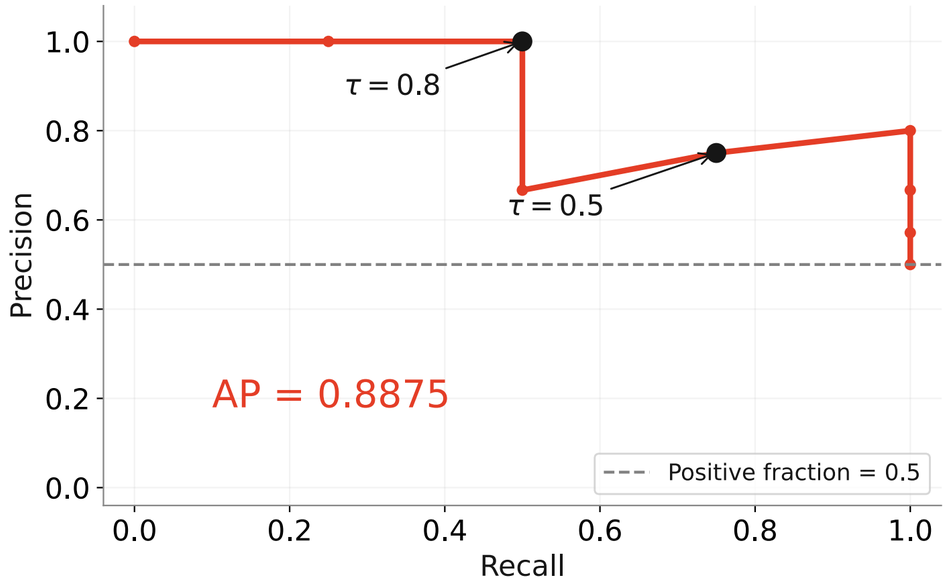
The PR curve focuses on positive predictions

Sweep the same thresholds, now plotting precision vertically against recall horizontally.

PR directly shows how reliable alerts remain as more actual positives are detected. It is especially informative when positives are rare.

$$\mathrm{AP} = \sum_k (R_k - R_{k-1}) P_k$$

Average precision (AP) weights precision by the gain in recall. Here AP = 0.8875. This uses a step-based convention, not trapezoidal PR area.



Same cases; positive fraction = 4/8 = 0.5. Points are linked in threshold order. The zero-alert endpoint uses precision = 1 for plotting.

Precision depends on class prevalence: compare PR results on comparable populations.

Choose the metric to match the decision

If only 10 of 1,000 cases are positive, always predicting 0 gives 99% accuracy but detects none of the events.

$$\text{Accuracy} = 0.99, \quad \text{Recall} = 0$$

Report class frequencies, the confusion matrix and complementary metrics. Choose a threshold on validation, then freeze it before the final test.

Question	Useful view
How many decisions are right?	Accuracy + baseline
Are alerts reliable?	Precision
Are events missed?	Recall
Balance precision and recall?	F1
How good is score ranking?	ROC–AUC / PR–AP
Are probabilities reliable?	Log loss + calibration

No single metric describes every aspect of a classifier.

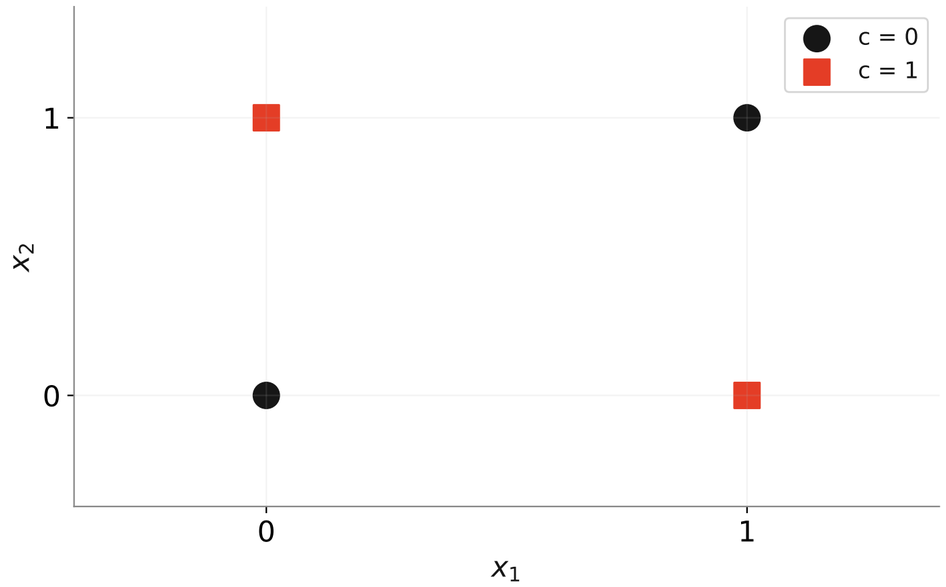
When a line is insufficient, change the representation

In XOR, positives occupy opposite corners. No single straight line separates the classes.

Introduce $\phi(x) = x_1 + x_2 - 2x_1x_2$. On the four binary inputs, it is 1 for class 1 and 0 for class 0.

$$p = \sigma(a\phi(x) + b)$$

Neural networks learn intermediate features instead of requiring us to specify each one.



With ϕ, a = 2 and b = −1 already classify all four inputs correctly at threshold 0.5.

Nonlinear features allow nonlinear boundaries in the original inputs.

Questions on classification and learning

01 **Connect model and task**
Why is logistic regression a classifier? Explain the route from x to p to a predicted label.

02 **Recover the gradient**
Explain the two factors in $(p - c)x$.

03 **Evaluate a threshold**
Which cases form the denominators of precision and recall?

04 **Evaluate a ranking**
What does ROC–AUC summarise? Does it choose the threshold?

$$x \longrightarrow z \longrightarrow p \longrightarrow \hat{c}$$

$$\ell \longrightarrow \nabla J \longrightarrow \theta^{+}$$

The loss trains the probability model. The threshold converts probabilities into class decisions.

For $p = 0.7$, $\tau = 0.8$ and observed label $c = 1$: what class is predicted, and which type of error occurs?

Define → derive → train → evaluate → decide.